

Praktikum 4: CPU Scheduling

Informasi Praktikum

- **Mata Kuliah:** Sistem Operasi
 - **Sistem Operasi:** Linux Ubuntu
 - **Bahasa Pemrograman:** C
 - **Dosen Pengampu:** Triawan Adi Cahyanto, M.Kom
 - **Asisten Dosen:** Atidhira Habibillah dan Taqiyyuddin
-

Pendahuluan

Dalam praktikum ini, kita akan belajar membuat program simulasi penjadwalan CPU menggunakan bahasa C di Linux. Mahasiswa akan membuat program yang meniru cara sistem operasi memilih proses mana yang harus dijalankan terlebih dahulu.

Bayangkan CPU sebagai kasir di supermarket, dan proses-proses adalah pelanggan yang mengantri. Algoritma penjadwalan adalah aturan yang menentukan urutan pelanggan mana yang dilayani terlebih dahulu.

Persiapan Praktikum

Tools yang Dibutuhkan:

1. **Sistem Operasi Linux (Debian/Ubuntu)**
2. **Text Editor** (bisa pilih salah satu):
 - nano (sederhana, cocok untuk pemula)
 - vim
 - gedit
 - VS Code
3. **Compiler GCC** (untuk mengcompile program C)

Instalasi GCC (jika belum ada):

```
sudo apt update
sudo apt install build-essential
```

Cek apakah GCC sudah terinstall:

```
gcc --version
```

Topik 1: Simulasi Algoritma FCFS (First Come First Served)

Tujuan:

Membuat program yang mensimulasikan algoritma FCFS, yaitu proses yang datang pertama akan dilayani pertama.

Konsep:

FCFS seperti antrian di kasir supermarket - siapa yang datang duluan, dia yang dilayani duluan.

Langkah-langkah:

1. Buat file program baru

```
nano fcfs.c
```

2. Ketik kode program berikut:

```
#include <stdio.h>

// Struktur untuk menyimpan data proses
struct Process {
    int pid;           // ID Proses
    int arrival_time;  // Waktu kedatangan
    int burst_time;    // Waktu eksekusi
    int waiting_time;  // Waktu menunggu
    int turnaround_time; // Waktu total (dari datang sampai selesai)
    int completion_time; // Waktu selesai
};

int main() {
    int n; // Jumlah proses

    printf("=== SIMULASI ALGORITMA FCFS ===\n");
    printf("Masukkan jumlah proses: ");
    scanf("%d", &n);

    struct Process proc[n];

    // Input data proses
    printf("\n");
    for(int i = 0; i < n; i++) {
        proc[i].pid = i + 1;
        printf("Proses P%d\n", proc[i].pid);
        printf("  Arrival Time: ");
        scanf("%d", &proc[i].arrival_time);
        printf("  Burst Time: ");
        scanf("%d", &proc[i].burst_time);
    }
}
```

```

        printf("\n");
    }

    // Hitung completion time, waiting time, dan turnaround time
    int current_time = 0;

    for(int i = 0; i < n; i++) {
        // Jika CPU idle, loncat ke waktu kedatangan proses
        if(current_time < proc[i].arrival_time) {
            current_time = proc[i].arrival_time;
        }

        proc[i].completion_time = current_time + proc[i].burst_time;
        proc[i].turnaround_time = proc[i].completion_time - proc[i].arrival_time;
        proc[i].waiting_time = proc[i].turnaround_time - proc[i].burst_time;

        current_time = proc[i].completion_time;
    }

    // Tampilkan hasil
    printf("\n=== HASIL PENJADWALAN FCFS ===\n");
    printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
    printf("---\t--\t--\t--\t--\t--\n");

    float total_tat = 0, total_wt = 0;

    for(int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
            proc[i].pid,
            proc[i].arrival_time,
            proc[i].burst_time,
            proc[i].completion_time,
            proc[i].turnaround_time,
            proc[i].waiting_time);

        total_tat += proc[i].turnaround_time;
        total_wt += proc[i].waiting_time;
    }

    printf("\n");
    printf("Average Turnaround Time: %.2f\n", total_tat / n);
    printf("Average Waiting Time: %.2f\n", total_wt / n);

    // Tampilkan Gantt Chart sederhana
    printf("\n=== GANTT CHART ===\n");
    printf("|");
    for(int i = 0; i < n; i++) {
        printf(" P%d |", proc[i].pid);
    }
    printf("\n");
    printf("0");
    for(int i = 0; i < n; i++) {
        printf("    %d", proc[i].completion_time);
    }
}

```

```
printf("\n");  
  
return 0;  
}
```

3. Compile dan jalankan:

```
gcc fcfs.c -o fcfs  
./fcfs
```

Topik 2: Simulasi Algoritma SJF (Shortest Job First)

Tujuan:

Membuat program yang mensimulasikan algoritma SJF, yaitu proses dengan waktu eksekusi terpendek dilayani terlebih dahulu.

Konsep:

SJF seperti kasir supermarket yang mendahulukan pembeli dengan belanjaan paling sedikit supaya antrian cepat berkurang.

Langkah-langkah:

1. Buat file program baru:

```
nano sjf.c
```

2. Ketik kode program berikut:

```
#include <stdio.h>  
#include <stdbool.h>  
  
struct Process {  
    int pid;  
    int arrival_time;  
    int burst_time;  
    int waiting_time;  
    int turnaround_time;  
    int completion_time;  
    bool is_completed;  
};  
  
int main() {  
    int n;
```

```

printf("=== SIMULASI ALGORITMA SJF (Non-Preemptive) ===\n");
printf("Masukkan jumlah proses: ");
scanf("%d", &n);

struct Process proc[n];

// Input data proses
printf("\n");
for(int i = 0; i < n; i++) {
    proc[i].pid = i + 1;
    proc[i].is_completed = false;
    printf("Proses P%d\n", proc[i].pid);
    printf("  Arrival Time: ");
    scanf("%d", &proc[i].arrival_time);
    printf("  Burst Time: ");
    scanf("%d", &proc[i].burst_time);
    printf("\n");
}

int current_time = 0;
int completed = 0;
int sequence[n]; // Menyimpan urutan eksekusi

// Proses penjadwalan SJF
while(completed < n) {
    int shortest_job = -1;
    int min_burst = 99999;

    // Cari proses dengan burst time terpendek yang sudah datang
    for(int i = 0; i < n; i++) {
        if(!proc[i].is_completed &&
            proc[i].arrival_time <= current_time &&
            proc[i].burst_time < min_burst) {
            min_burst = proc[i].burst_time;
            shortest_job = i;
        }
    }

    // Jika tidak ada proses yang ready, loncat ke waktu proses berikutnya
    if(shortest_job == -1) {
        current_time++;
        continue;
    }

    // Eksekusi proses terpilih
    sequence[completed] = shortest_job;
    current_time += proc[shortest_job].burst_time;
    proc[shortest_job].completion_time = current_time;
    proc[shortest_job].turnaround_time =
        proc[shortest_job].completion_time - proc[shortest_job].arrival_time;
    proc[shortest_job].waiting_time =
        proc[shortest_job].turnaround_time - proc[shortest_job].burst_time;
    proc[shortest_job].is_completed = true;
}

```

```

        completed++;
    }

    // Tampilkan hasil
    printf("\n=== HASIL PENJADWALAN SJF ===\n");
    printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
    printf("---\t--\t--\t--\t--\t--\n");

    float total_tat = 0, total_wt = 0;

    for(int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
            proc[i].pid,
            proc[i].arrival_time,
            proc[i].burst_time,
            proc[i].completion_time,
            proc[i].turnaround_time,
            proc[i].waiting_time);

        total_tat += proc[i].turnaround_time;
        total_wt += proc[i].waiting_time;
    }

    printf("\n");
    printf("Average Turnaround Time: %.2f\n", total_tat / n);
    printf("Average Waiting Time: %.2f\n", total_wt / n);

    // Tampilkan urutan eksekusi
    printf("\n=== URUTAN EKSEKUSI ===\n");
    printf("Proses dieksekusi dalam urutan: ");
    for(int i = 0; i < n; i++) {
        printf("P%d ", proc[sequence[i]].pid);
        if(i < n-1) printf("-> ");
    }
    printf("\n");

    return 0;
}

```

3. Compile dan jalankan:

```

gcc sjf.c -o sjf
./sjf

```

Topik 3: Simulasi Algoritma Round Robin

Tujuan:

Membuat program yang mensimulasikan algoritma Round Robin, yaitu setiap proses mendapat jatah waktu yang sama (time quantum).

Konsep:

Round Robin seperti bermain game dengan teman - setiap orang main bergantian dengan waktu yang sama, misalnya 5 menit per orang.

Langkah-langkah:

1. Buat file program baru:

```
nano round_robin.c
```

2. Ketik kode program berikut:

```
#include <stdio.h>
#include <stdbool.h>

struct Process {
    int pid;
    int arrival_time;
    int burst_time;
    int remaining_time;
    int waiting_time;
    int turnaround_time;
    int completion_time;
};

int main() {
    int n, quantum;

    printf("=== SIMULASI ALGORITMA ROUND ROBIN ===\n");
    printf("Masukkan jumlah proses: ");
    scanf("%d", &n);
    printf("Masukkan time quantum: ");
    scanf("%d", &quantum);

    struct Process proc[n];

    // Input data proses
    printf("\n");
    for(int i = 0; i < n; i++) {
        proc[i].pid = i + 1;
        printf("Proses P%d\n", proc[i].pid);
        printf("  Arrival Time: ");
        scanf("%d", &proc[i].arrival_time);
        printf("  Burst Time: ");
        scanf("%d", &proc[i].burst_time);
```

```

        proc[i].remaining_time = proc[i].burst_time;
        printf("\n");
    }

    int current_time = 0;
    int completed = 0;
    int queue[100], front = 0, rear = 0;
    bool in_queue[n];

    // Inisialisasi
    for(int i = 0; i < n; i++) {
        in_queue[i] = false;
    }

    // Masukkan proses pertama yang sudah datang ke queue
    for(int i = 0; i < n; i++) {
        if(proc[i].arrival_time <= current_time) {
            queue[rear++] = i;
            in_queue[i] = true;
        }
    }

    printf("\n=== PROSES EKSEKUSI ===\n");

    // Proses penjadwalan Round Robin
    while(completed < n) {
        if(front == rear) {
            // Queue kosong, loncat ke proses berikutnya yang datang
            current_time++;
            for(int i = 0; i < n; i++) {
                if(!in_queue[i] && proc[i].arrival_time <= current_time &&
                    proc[i].remaining_time > 0) {
                    queue[rear++] = i;
                    in_queue[i] = true;
                }
            }
            continue;
        }

        int idx = queue[front++];
        in_queue[idx] = false;

        printf("Waktu %d: Menjalankan P%d", current_time, proc[idx].pid);

        if(proc[idx].remaining_time <= quantum) {
            // Proses selesai dalam quantum ini
            current_time += proc[idx].remaining_time;
            proc[idx].remaining_time = 0;
            proc[idx].completion_time = current_time;
            proc[idx].turnaround_time =
                proc[idx].completion_time - proc[idx].arrival_time;
            proc[idx].waiting_time =
                proc[idx].turnaround_time - proc[idx].burst_time;
            completed++;
        }
    }

```



```

        printf(" -> Selesai pada waktu %d\n", current_time);
    } else {
        // Proses belum selesai
        current_time += quantum;
        proc[idx].remaining_time -= quantum;
        printf(" -> Sisa waktu: %d\n", proc[idx].remaining_time);
    }

    // Masukkan proses baru yang sudah datang ke queue
    for(int i = 0; i < n; i++) {
        if(!in_queue[i] && proc[i].arrival_time <= current_time &&
            proc[i].remaining_time > 0) {
            queue[rear++] = i;
            in_queue[i] = true;
        }
    }

    // Jika proses belum selesai, masukkan kembali ke queue
    if(proc[idx].remaining_time > 0) {
        queue[rear++] = idx;
        in_queue[idx] = true;
    }
}

// Tampilkan hasil
printf("\n=== HASIL PENJADWALAN ROUND ROBIN ===\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
printf("---\t--\t--\t--\t---\t--\n");

float total_tat = 0, total_wt = 0;

for(int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        proc[i].pid,
        proc[i].arrival_time,
        proc[i].burst_time,
        proc[i].completion_time,
        proc[i].turnaround_time,
        proc[i].waiting_time);

    total_tat += proc[i].turnaround_time;
    total_wt += proc[i].waiting_time;
}

printf("\n");
printf("Average Turnaround Time: %.2f\n", total_tat / n);
printf("Average Waiting Time: %.2f\n", total_wt / n);

return 0;
}

```

3. Compile dan jalankan:

```
gcc round_robin.c -o round_robin
./round_robin
```

Topik 4: Perbandingan Algoritma

Tujuan:

Membuat program yang membandingkan ketiga algoritma (FCFS, SJF, Round Robin) dengan data yang sama.

Langkah-langkah:

1. Buat file program:

```
nano compare.c
```

2. Ketik kode program yang menggabungkan ketiga algoritma

```
#include <stdio.h>
#include <stdbool.h>
#include <string.h>

// Struktur untuk menyimpan data proses
struct Process {
    int pid;
    int arrival_time;
    int burst_time;
    int waiting_time;
    int turnaround_time;
    int completion_time;
    int remaining_time;
    bool is_completed;
};

// Struktur untuk menyimpan hasil algoritma
struct AlgorithmResult {
    char name[30];
    float avg_waiting_time;
    float avg_turnaround_time;
    int total_context_switches;
};

// Fungsi untuk reset data proses
void reset_processes(struct Process proc[], struct Process backup[], int n) {
    for(int i = 0; i < n; i++) {
        proc[i] = backup[i];
        proc[i].waiting_time = 0;
    }
}
```

```

        proc[i].turnaround_time = 0;
        proc[i].completion_time = 0;
        proc[i].remaining_time = proc[i].burst_time;
        proc[i].is_completed = false;
    }
}

// Fungsi untuk menampilkan hasil detail
void display_results(struct Process proc[], int n, char* algorithm_name) {
    printf("\n=====\\n");
    printf("    HASIL ALGORITMA %s\\n", algorithm_name);
    printf("=====\\n");
    printf("PID\\tAT\\tBT\\tCT\\tTAT\\tWT\\n");
    printf("---\\t--\\t--\\t--\\t---\\t--\\n");

    float total_tat = 0, total_wt = 0;

    for(int i = 0; i < n; i++) {
        printf("P%d\\t%d\\t%d\\t%d\\t%d\\t%d\\n",
            proc[i].pid,
            proc[i].arrival_time,
            proc[i].burst_time,
            proc[i].completion_time,
            proc[i].turnaround_time,
            proc[i].waiting_time);

        total_tat += proc[i].turnaround_time;
        total_wt += proc[i].waiting_time;
    }

    printf("\\n");
    printf("Average Turnaround Time: %.2f\\n", total_tat / n);
    printf("Average Waiting Time: %.2f\\n", total_wt / n);
}

// =====
// ALGORITMA FCFS (First Come First Served)
// =====
struct AlgorithmResult fcfs_scheduling(struct Process proc[], int n) {
    struct AlgorithmResult result;
    strcpy(result.name, "FCFS");
    result.total_context_switches = n - 1;

    int current_time = 0;
    float total_tat = 0, total_wt = 0;

    for(int i = 0; i < n; i++) {
        // Jika CPU idle, loncat ke waktu kedatangan proses
        if(current_time < proc[i].arrival_time) {
            current_time = proc[i].arrival_time;
        }

        proc[i].completion_time = current_time + proc[i].burst_time;
        proc[i].turnaround_time = proc[i].completion_time - proc[i].arrival_time;
    }
}

```

```

        proc[i].waiting_time = proc[i].turnaround_time - proc[i].burst_time;

        current_time = proc[i].completion_time;

        total_tat += proc[i].turnaround_time;
        total_wt += proc[i].waiting_time;
    }

    result.avg_turnaround_time = total_tat / n;
    result.avg_waiting_time = total_wt / n;

    display_results(proc, n, "FCFS");

    return result;
}

// =====
// ALGORITMA SJF (Shortest Job First)
// =====
struct AlgorithmResult sjf_scheduling(struct Process proc[], int n) {
    struct AlgorithmResult result;
    strcpy(result.name, "SJF");

    int current_time = 0;
    int completed = 0;
    float total_tat = 0, total_wt = 0;
    int context_switches = 0;

    while(completed < n) {
        int shortest_job = -1;
        int min_burst = 99999;

        // Cari proses dengan burst time terpendek yang sudah datang
        for(int i = 0; i < n; i++) {
            if(!proc[i].is_completed &&
                proc[i].arrival_time <= current_time &&
                proc[i].burst_time < min_burst) {
                min_burst = proc[i].burst_time;
                shortest_job = i;
            }
        }

        // Jika tidak ada proses yang ready, loncat ke waktu proses berikutnya
        if(shortest_job == -1) {
            current_time++;
            continue;
        }

        // Eksekusi proses terpilih
        current_time += proc[shortest_job].burst_time;
        proc[shortest_job].completion_time = current_time;
        proc[shortest_job].turnaround_time =
            proc[shortest_job].completion_time - proc[shortest_job].arrival_time;
        proc[shortest_job].waiting_time =

```

```

        proc[shortest_job].turnaround_time - proc[shortest_job].burst_time;
        proc[shortest_job].is_completed = true;

        total_tat += proc[shortest_job].turnaround_time;
        total_wt += proc[shortest_job].waiting_time;

        completed++;
        if(completed < n) context_switches++;
    }

    result.avg_turnaround_time = total_tat / n;
    result.avg_waiting_time = total_wt / n;
    result.total_context_switches = context_switches;

    display_results(proc, n, "SJF");

    return result;
}

// =====
// ALGORITMA ROUND ROBIN
// =====
struct AlgorithmResult round_robin_scheduling(struct Process proc[], int n, int
quantum) {
    struct AlgorithmResult result;
    sprintf(result.name, "Round Robin (Q=%d)", quantum);

    int current_time = 0;
    int completed = 0;
    float total_tat = 0, total_wt = 0;
    int context_switches = 0;

    int queue[100], front = 0, rear = 0;
    bool in_queue[n];

    // Inisialisasi
    for(int i = 0; i < n; i++) {
        in_queue[i] = false;
    }

    // Masukkan proses pertama yang sudah datang ke queue
    for(int i = 0; i < n; i++) {
        if(proc[i].arrival_time <= current_time) {
            queue[rear++] = i;
            in_queue[i] = true;
        }
    }

    // Proses penjadwalan Round Robin
    while(completed < n) {
        if(front == rear) {
            // Queue kosong, loncat ke proses berikutnya yang datang
            current_time++;
            for(int i = 0; i < n; i++) {

```

```

        if(!in_queue[i] && proc[i].arrival_time <= current_time &&
           proc[i].remaining_time > 0) {
            queue[rear++] = i;
            in_queue[i] = true;
        }
    }
    continue;
}

int idx = queue[front++];
in_queue[idx] = false;

if(proc[idx].remaining_time <= quantum) {
    // Proses selesai dalam quantum ini
    current_time += proc[idx].remaining_time;
    proc[idx].remaining_time = 0;
    proc[idx].completion_time = current_time;
    proc[idx].turnaround_time =
        proc[idx].completion_time - proc[idx].arrival_time;
    proc[idx].waiting_time =
        proc[idx].turnaround_time - proc[idx].burst_time;

    total_tat += proc[idx].turnaround_time;
    total_wt += proc[idx].waiting_time;

    completed++;
} else {
    // Proses belum selesai
    current_time += quantum;
    proc[idx].remaining_time -= quantum;
    context_switches++;
}

// Masukkan proses baru yang sudah datang ke queue
for(int i = 0; i < n; i++) {
    if(!in_queue[i] && proc[i].arrival_time <= current_time &&
       proc[i].remaining_time > 0) {
        queue[rear++] = i;
        in_queue[i] = true;
    }
}

// Jika proses belum selesai, masukkan kembali ke queue
if(proc[idx].remaining_time > 0) {
    queue[rear++] = idx;
    in_queue[idx] = true;
}

result.avg_turnaround_time = total_tat / n;
result.avg_waiting_time = total_wt / n;
result.total_context_switches = context_switches;

display_results(proc, n, result.name);

```

```

        return result;
    }

// Fungsi untuk menampilkan tabel perbandingan
void display_comparison(struct AlgorithmResult results[], int num_algorithms) {
    printf("\n\n");

    printf("=====\n");
    printf("                TABEL PERBANDINGAN ALGORITMA\n");
    printf("=====\n");
    printf("%-20s | %10s | %10s | %15s\n",
           "Algoritma", "Avg TAT", "Avg WT", "Context Switch");
    printf("-----\n");

    int best_tat = 0, best_wt = 0;
    float min_tat = results[0].avg_turnaround_time;
    float min_wt = results[0].avg_waiting_time;

    for(int i = 0; i < num_algorithms; i++) {
        printf("%-20s | %10.2f | %10.2f | %15d\n",
               results[i].name,
               results[i].avg_turnaround_time,
               results[i].avg_waiting_time,
               results[i].total_context_switches);

        if(results[i].avg_turnaround_time < min_tat) {
            min_tat = results[i].avg_turnaround_time;
            best_tat = i;
        }

        if(results[i].avg_waiting_time < min_wt) {
            min_wt = results[i].avg_waiting_time;
            best_wt = i;
        }
    }

    printf("=====\n");
    printf("\nKESIMPULAN:\n");
    printf("✓ Algoritma terbaik (Avg TAT terkecil): %s (%.2f)\n",
           results[best_tat].name, min_tat);
    printf("✓ Algoritma terbaik (Avg WT terkecil): %s (%.2f)\n",
           results[best_wt].name, min_wt);
}

// Fungsi untuk menampilkan grafik perbandingan ASCII
void display_bar_chart(struct AlgorithmResult results[], int num_algorithms) {
    printf("\n\n");

```

```

printf("=====\\n");
printf("          GRAFIK PERBANDINGAN AVERAGE WAITING TIME\\n");

printf("=====\\n");

// Cari nilai maksimum untuk scaling
float max_wt = 0;
for(int i = 0; i < num_algorithms; i++) {
    if(results[i].avg_waiting_time > max_wt) {
        max_wt = results[i].avg_waiting_time;
    }
}

// Tampilkan bar chart
for(int i = 0; i < num_algorithms; i++) {
    printf("%-20s |", results[i].name);

    int bar_length = (int)((results[i].avg_waiting_time / max_wt) * 50);
    for(int j = 0; j < bar_length; j++) {
        printf("█");
    }
    printf(" %.2f\\n", results[i].avg_waiting_time);
}

printf("=====\\n");
}

int main() {
    int n, quantum;

    printf("\\n");
    printf("=====\\n");
    printf("    PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING    \\n");
    printf("=====\\n");
    printf("    Program ini akan membandingkan 3 algoritma:    \\n");
    printf("    1. FCFS (First Come First Served)    \\n");
    printf("    2. SJF (Shortest Job First)    \\n");
    printf("    3. Round Robin    \\n");
    printf("=====\\n");
    printf("\\n");

    printf("Masukkan jumlah proses: ");
    scanf("%d", &n);

    printf("Masukkan time quantum untuk Round Robin: ");
    scanf("%d", &quantum);

    struct Process proc[n];
    struct Process backup[n]; // Backup data untuk reset

```



```

// Input data proses
printf("\n");
for(int i = 0; i < n; i++) {
    proc[i].pid = i + 1;
    printf("== Proses P%d ==\n", proc[i].pid);
    printf("Arrival Time: ");
    scanf("%d", &proc[i].arrival_time);
    printf("Burst Time: ");
    scanf("%d", &proc[i].burst_time);
    printf("\n");

    // Simpan backup
    backup[i] = proc[i];
}

struct AlgorithmResult results[3];

// Jalankan FCFS
printf("\n");
printf("===== \n");
printf("MENJALANKAN ALGORITMA FCFS \n");
printf("===== \n");
reset_processes(proc, backup, n);
results[0] = fcfs_scheduling(proc, n);

printf("\nTekan Enter untuk melanjutkan...");
getchar();
getchar();

// Jalankan SJF
printf("\n");
printf("===== \n");
printf("MENJALANKAN ALGORITMA SJF \n");
printf("===== \n");
reset_processes(proc, backup, n);
results[1] = sjf_scheduling(proc, n);

printf("\nTekan Enter untuk melanjutkan...");
getchar();

// Jalankan Round Robin
printf("\n");
printf("===== \n");
printf("MENJALANKAN ALGORITMA ROUND ROBIN \n");
printf("===== \n");
reset_processes(proc, backup, n);
results[2] = round_robin_scheduling(proc, n, quantum);

printf("\nTekan Enter untuk melihat perbandingan...");
getchar();

// Tampilkan perbandingan
display_comparison(results, 3);

```

```
// Tampilkan grafik
display_bar_chart(results, 3);

printf("\n");
printf("PROGRAM SELESAI\n");
printf("\n");
return 0;
}
```

3. Jalankan dan bandingkan hasilnya

Tugas Praktikum

Tugas 1: Eksperimen dengan Data Berbeda

Jalankan setiap algoritma dengan data berikut dan catat hasilnya:

Data Set 1:

- P1: AT=0, BT=10
- P2: AT=1, BT=5
- P3: AT=2, BT=8

Data Set 2:

- P1: AT=0, BT=5
- P2: AT=1, BT=3
- P3: AT=2, BT=8
- P4: AT=3, BT=6

Tugas 2: Analisis Eksperimen ketiga algoritma

Jawab pertanyaan berikut:

1. Algoritma mana yang memberikan Average Waiting Time terkecil?
2. Mengapa hasil algoritma berbeda-beda?
3. Kapan sebaiknya menggunakan algoritma FCFS?
4. Kapan sebaiknya menggunakan algoritma SJF?
5. Kapan sebaiknya menggunakan algoritma Round Robin?

Tugas 3: Modifikasi Program

Tambahkan fitur berikut pada salah satu program:

1. Tampilkan Gantt Chart yang lebih detail
2. Simpan hasil ke file text
3. Baca input dari file

Tugas 4: Eksperimen dengan data berbeda untuk perbandingan algoritma

Test Case 1: Proses dengan burst time bervariasi

Jumlah proses: 4
Time quantum: 3

- P1: AT=0, BT=24
- P2: AT=1, BT=3
- P3: AT=2, BT=3
- P4: AT=3, BT=6

Test Case 2: Proses datang bersamaan

Jumlah proses: 5
Time quantum: 4

- P1: AT=0, BT=10
- P2: AT=0, BT=5
- P3: AT=0, BT=8
- P4: AT=0, BT=12
- P5: AT=0, BT=6

Test Case 3: Proses datang dengan interval

Jumlah proses: 5
Time quantum: 2

- P1: AT=0, BT=8
- P2: AT=2, BT=4
- P3: AT=4, BT=9
- P4: AT=6, BT=5
- P5: AT=8, BT=3

Tugas 5: Analisis Eksperimen perbandingan algoritma

Setelah menjalankan program dengan berbagai test case, jawab pertanyaan berikut:

1. Analisis Performa

- Dalam kondisi apa FCFS memberikan hasil terbaik?
- Mengapa SJF hampir selalu memberikan Average WT terkecil?
- Apa trade-off dari Round Robin?

2. Context Switching

- Algoritma mana yang paling banyak melakukan context switch?
- Bagaimana pengaruh quantum terhadap jumlah context switch di Round Robin?
- Coba jalankan Round Robin dengan quantum 2, 4, 8, 16 - apa yang terjadi?

3. Fairness

- Algoritma mana yang paling "adil" untuk semua proses?
- Proses mana yang paling dirugikan di algoritma FCFS?
- Apakah ada kemungkinan starvation di SJF?

4. Real World Application

- Untuk web server yang melayani banyak request kecil, algoritma mana yang cocok?
- Untuk render video (task besar), algoritma mana yang cocok?
- Untuk OS desktop (interactive), algoritma mana yang cocok?

Perhatian:

Khusus soal terkait analisis, silakan jawab dan lampirkan bukti nyata/konkret terkait data pendukung jawaban yang Anda tulis maupun berbentuk luaran (hasil run kode program), jadi tidak sekedar menjawab dengan menulis narasi saja.

Tips Praktikum

1. Jika program error saat compile:

- Periksa apakah ada tanda titik koma (;) yang kurang
- Pastikan semua kurung kurawal { } berpasangan
- Cek ejaan nama variabel

2. Cara debug program:

```
gcc -g nama_file.c -o nama_program
gdb ./nama_program
```

3. Cara menyimpan output ke file:

```
./fcfs > hasil.txt
```

4. Membersihkan terminal:

```
clear
```